



UniNotify

A Serverless Student Announcement & Notification System on AWS

CE308 – Cloud Computing | Project Report | GIKI Spring 2026

Member Name	Registration Number
Abdul Wadood Khan	2023029
Ashir Siddiqui	2023385
Rayyan Mehmood	2023602
Syed Murtaza Salman	2023694

1. Introduction

UniNotify is a fully serverless, cloud-native announcement system built entirely on Amazon Web Services (AWS). The system allows faculty and administrators at GIKI to send structured announcements to targeted student audiences through a clean web interface. Announcements are stored permanently in a NoSQL database, delivered instantly via email notifications, and the entire backend operates without a single managed server.

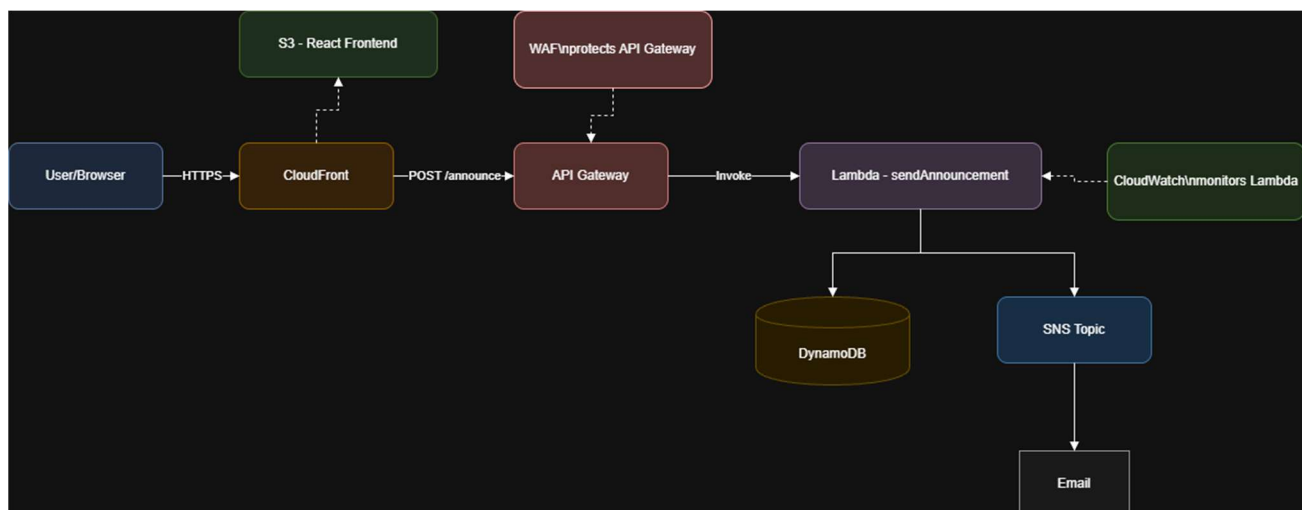
This project was developed as part of CE308 Cloud Computing to demonstrate practical mastery of AWS services, serverless architecture, REST API design, NoSQL databases, cloud security, and content delivery — all topics explicitly covered in the CE308 weekly syllabus.

2. Problem Statement

GIKI departments currently rely on WhatsApp groups and informal email threads for announcements. These methods are unstructured, offer no delivery guarantee, cannot target specific audiences, and leave no persistent record of communications. UniNotify replaces this with a structured, cloud-backed solution where every announcement is stored permanently with a unique ID and timestamp, delivered reliably to the correct audience, and accessible through a professional web interface — at zero cost under AWS Free Tier usage.

3. System Architecture

UniNotify follows a serverless, event-driven architecture composed of six AWS services connected in a linear pipeline. There is no EC2 instance, no web server, and no database engine to manage. Every component is fully managed by AWS and scales automatically.



3.1 Data Flow

The complete flow for a single announcement submission works as follows:

1. Faculty opens the CloudFront URL in their browser.
2. CloudFront fetches the React application files from the private S3 bucket and serves them.
3. Faculty fills in the title, selects an audience (e.g. CE Department), types a message, and clicks Send.
4. The React app sends a POST request to the API Gateway endpoint /announce with the form data as JSON.
5. WAF inspects the request, checks against the Core Rule Set, and passes it through if safe.
6. API Gateway invokes the Lambda function sendAnnouncement with the full request.
7. Lambda validates the input, writes the record to DynamoDB, and publishes to the correct SNS topic.
8. SNS emails every confirmed subscriber of that topic within seconds.
9. CloudWatch automatically logs the Lambda invocation and updates the dashboard.

3.2 Service Overview

AWS Service	Role in UniNotify	CE308 Topic
S3	Hosts the React frontend as static files	SaaS model (Week 2)
CloudFront	CDN — serves frontend globally, secures S3	Content delivery (Week 12)
API Gateway	REST endpoint that receives form submissions	REST protocols (Week 5)
Lambda	Serverless backend — validates, stores, notifies	Serverless / PaaS (Week 13)
DynamoDB	NoSQL database storing all announcements	Non-relational DB (Week 10)
SNS	Sends targeted email notifications per audience	Event-driven arch (Week 4)
IAM	Least-privilege permissions for Lambda	Security (Week 11)
CloudWatch	Logs, dashboard, and error alarms	Monitoring (Week 11)
WAF	Firewall protecting the API Gateway	Cloud security (Week 11)

4. Implementation

4.1 S3 & CloudFront — Frontend Hosting

The React frontend was built using Create React App and compiled into a static build folder. This folder was uploaded to a private S3 bucket with all public access blocked. A CloudFront distribution was created pointing to this bucket using Origin Access Control (OAC), which generates a bucket policy allowing only CloudFront to read from S3. Users access the application exclusively through the CloudFront URL — they never interact with S3 directly. This architecture is more secure than direct S3 hosting and delivers the frontend faster globally through CloudFront's edge network.

Objects (8)
🔄
📄 Copy S3 URI
📄 Copy URL
⬇️ Download
🔗 Open ↗️
Delete

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	index.html	html	April 30, 2026, 20:22:43 (UTC+05:00)	644.0 B	Standard
☐	logo192.png	png	April 30, 2026, 20:22:44 (UTC+05:00)	5.2 KB	Standard
☐	logo512.png	png	April 30, 2026, 20:22:47 (UTC+05:00)	9.4 KB	Standard
☐	manifest.json	json	April 30, 2026, 20:22:49 (UTC+05:00)	492.0 B	Standard
☐	robots.txt	txt	April 30, 2026, 20:22:50 (UTC+05:00)	67.0 B	Standard
☐	static/	Folder	-	-	-

Distributions (1)
Info
🔄
Enable
Disable
Delete

Filter type
All distributions ▼

☐	ID	Status	Descrip...	Type	Domai...	Alterna..
☐	E1O13ODGTQXTFR	Enabled	-	Standard	d13lr...	-

4.2 API Gateway — REST Endpoint

A REST API named UniNotifyAPI was created in API Gateway with a single resource /announce and a POST method. Lambda Proxy Integration was enabled so that the full HTTP request is forwarded to Lambda unchanged. CORS was enabled on the resource to allow the React frontend running on a different domain (CloudFront) to make requests to the API. The API was deployed to a production stage named prod, generating the invoke URL used by the frontend.

The screenshot displays the AWS API Gateway console interface. The top section, titled "Resources", shows a tree view on the left with a "Create resource" button and a list containing "/" and "/announce". The "/announce" resource is selected, showing "OPTIONS" and "POST" methods. To the right, the "Resource details" panel shows the path "/announce" and resource ID "iygzxc", with "Delete" and "Update documentation" buttons. Below this, the "Methods (2)" section lists "OPTIONS" (Mock integration) and "POST" (Lambda integration), both with "None" authorization. The bottom section, titled "Stages", shows a list with "prod" selected. The "Stage details" panel for "prod" shows the stage name, rate limit of 10000, cache cluster as "Inactive", default method-level caching as "Inactive", and the invoke URL: <https://ms84vqng5l.execute-api.eu-north-1.amazonaws.com/prod>.

Resources

[API actions](#)

[Create resource](#)

- /
- /announce**
 - OPTIONS
 - POST

Resource details

[Delete](#) [Update documentation](#)

Path: /announce Resource ID: iygzxc

Methods (2)

[Delete](#)

	Method type ▲	Integration type ▼	Authorization
☐	OPTIONS	Mock	None
☐	POST	Lambda	None

Stages

prod

Stage details [Info](#)

Stage name: prod	Rate Info : 10000
Cache cluster Info : Inactive	Burst Info : 5000
Default method-level caching: Inactive	

Invoke URL: <https://ms84vqng5l.execute-api.eu-north-1.amazonaws.com/prod>

4.3 Lambda — Serverless Backend

The Lambda function `sendAnnouncement` was written in Python 3.12 and serves as the entire backend of the application. It performs three operations on every invocation: input validation, DynamoDB write, and SNS publish. The function uses a `TOPIC_MAP` dictionary to route announcements to the correct SNS topic based on the audience selected in the form. This means selecting CE Department sends only to CE students, Faculty sends only to faculty, and so on. The function handles CORS preflight `OPTIONS` requests and returns appropriate error responses for invalid input.

The screenshot displays the AWS Lambda console interface for the `sendAnnouncement` function. The top navigation bar includes tabs for **Code**, **Test**, **Monitor**, **Configuration**, **Aliases**, and **Versions**. The **Code source** tab is active, showing a code editor with the `lambda_function.py` file. The code includes imports for `json`, `boto3`, and `uuid`, and uses `boto3` to interact with `dynamodb` and `sns`. The `TABLE_NAME` is set to `'Announcements'`. Below the code editor, the **Runtime settings** section shows the function is configured with `Python 3.12` runtime, `lambda_function.lambda_handler` as the handler, and `x86_64` architecture. A green banner at the bottom indicates the function execution was successful, showing the response body: `{ "statusCode": 200, "headers": { "Access-Control-Allow-Origin": "*" }, "body": "{\"message\": \"Announcement sent to All Students!\"}" }`.

Code source Info [Open in Visual Studio Code](#) [Update](#)

EXPLORER

- SENDANNOUNCEMENT
 - lambda_function.py

DEPLOY [Deploy \(Ctrl+Shift+U\)](#)

```
1 import json
2 import boto3
3 import uuid
4 from datetime import datetime
5
6 dynamodb = boto3.resource('dynamodb')
7 sns = boto3.client('sns')
8
9 TABLE_NAME = 'Announcements'
```

Runtime settings Info [Edit](#) [Edit runtime management configuration](#)

Runtime Python 3.12

Handler [Info](#) lambda_function.lambda_handler

Architecture [Info](#) x86_64

[Lambda](#) > [Functions](#) > sendAnnouncement

✓ Executing function: succeeded ([logs](#))

▼ Details

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*"
  },
  "body": "{\"message\": \"Announcement sent to All Students!\"}"
}
```

4.4 DynamoDB — Persistent Storage

A DynamoDB table named Announcements was created with on-demand capacity mode. The partition key is id (a UUID generated by Lambda on each invocation) and the sort key is created_at (an ISO 8601 UTC timestamp). Every announcement submitted through the form is stored as an item in this table with all five fields: id, created_at, title, message, and audience. On-demand mode was chosen because it scales automatically with no capacity planning required and is cost-free under Free Tier usage levels.

Table: Announcements - Items returned (11)


Actions ▼
Create item

Scan started on May 09, 2026, 12:13:32

< 1 > 

☐	id (String) ▼	created_at (String) ▼	audience ▼	
☐	8626b942-b4fe-4cfd-...	2026-05-05T07:07:09.7...	All Students	r
☐	4de7a3c1-f297-412a-...	2026-05-09T07:11:32.8...	All Students	H
☐	9e640bc7-8d8b-4f80-...	2026-04-28T18:35:32.6...	All Students	T
☐	2c8b5afe-9998-4d80-...	2026-05-05T07:09:12.1...	All Students	M
☐	1c3d509e-6b64-44a0-...	2026-05-07T13:52:07.5...	Faculty	T
☐	22ef4e17-7f9d-49b5-...	2026-05-08T17:38:58.7...	CE Depart...	l

4.5 SNS — Targeted Email Notifications

Five SNS topics were created corresponding to the five audience options in the form: uninotify-all-students, uninotify-ce-department, uninotify-cs-department, uninotify-final-year, and uninotify-faculty. Each topic has email subscriptions for the relevant recipients. Every subscriber confirmed their subscription via the AWS confirmation email. When Lambda calls sns.publish() with the correct topic ARN, SNS delivers the announcement email to all confirmed subscribers of that topic within seconds.

The screenshot displays the AWS SNS console interface. The top section, titled 'Topics (7)', includes buttons for 'Edit', 'Delete', 'Publish message', and 'Create topic'. Below this is a search bar and a table of topics. The table has columns for Name, Type, and ARN. The topics listed are: uninotify-all-students, uninotify-announcements, uninotify-ce-department, uninotify-cs-department, uninotify-faculty, and uninotify-final-year, all of type 'Standard'. The bottom section, titled 'Subscriptions (2)', includes buttons for 'Edit', 'Delete', 'Request confirmation', 'Confirm subscription', and 'Create subscription'. It also has a search bar and a table of subscriptions. The table has columns for ID, Endpoint, Status, and Protocol. The subscriptions listed are: 1f777629-9b49-4437-b... (Endpoint: u2023385@giki.edu.pk, Status: Confirmed, Protocol: EMAIL) and 913c8635-447e-4caa-9c... (Endpoint: u2023694@giki.edu.pk, Status: Confirmed, Protocol: EMAIL).

Name	Type	ARN
uninotify-all-students	Standard	arn:aws:sns:eu-north-1:4124901608...
uninotify-announcements	Standard	arn:aws:sns:eu-north-1:4124901608...
uninotify-ce-department	Standard	arn:aws:sns:eu-north-1:4124901608...
uninotify-cs-department	Standard	arn:aws:sns:eu-north-1:4124901608...
uninotify-faculty	Standard	arn:aws:sns:eu-north-1:4124901608...
uninotify-final-year	Standard	arn:aws:sns:eu-north-1:4124901608...

ID	Endpoint	Status	Protocol
1f777629-9b49-4437-b...	u2023385@giki.edu.pk	Confirmed	EMAIL
913c8635-447e-4caa-9c...	u2023694@giki.edu.pk	Confirmed	EMAIL

4.6 IAM — Least Privilege Security

A custom IAM execution role named uninotify-lambda-role was created for Lambda. Rather than using broad managed policies, a least-privilege inline policy was attached that grants only the three permissions Lambda actually needs: dynamodb:PutItem on the Announcements table, sns:Publish on all five UniNotify topics, and CloudWatch Logs permissions. This follows the principle of least privilege from the AWS Shared Responsibility Model — Lambda cannot access any other DynamoDB table, SNS topic, or AWS service.

Permissions policies (2) Info

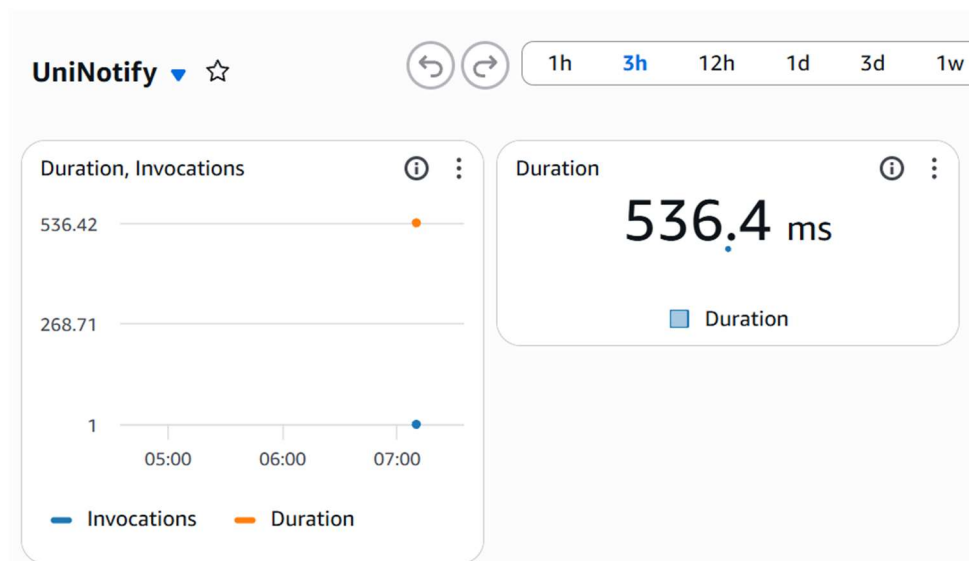
You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
uninotify-least-privilege	Customer inline	0
uninotify-policy	Customer inline	0

4.7 CloudWatch — Monitoring & Alarms

CloudWatch automatically captures logs from every Lambda invocation without any additional configuration, because Lambda writes to CloudWatch using the permissions in its IAM role. A custom dashboard named UniNotify was created showing Lambda invocations and duration over time. A CloudWatch Alarm was configured to monitor the Lambda error metric and trigger if errors exceed two within a five-minute window, publishing an alert to a separate SNS topic uninotify-alerts subscribed to the team's emails.



Log streams (5)

Delete

Create log stream

By default, we only load the most recent log streams.

Filter log streams or try prefix search

Exact match

Show expired

Log stream

Last event time

	2026/05/05/[\$LATEST]7a77b586ccfe4415b6842049243cfd88	2026-05-05 07:09:12 (UTC)
	2026/05/03/[\$LATEST]50ea6e76dee844d090bccfe403c05ad8	2026-05-03 20:54:05 (UTC)
	2026/04/30/[\$LATEST]98defb5ce5c5470b949acc6070190dfc	2026-04-30 17:57:33 (UTC)
	2026/04/28/[\$LATEST]f051f1f1a2ac41e8a5327c773b00406d	2026-04-28 18:35:33 (UTC)
	2026/04/27/[\$LATEST]cbe9e48a02734736afeb108f3276f2a6	2026-04-27 18:41:12 (UTC)

Alarms

UniNotify-Lambda-Errors

Actions

View

Explore related

Investigate

Count

Errors > 2 for 1 datapoints within 5 minutes - (2)

2

1

0

04:45

05:00

05:15

05:30

05:45

06:00

06:15

06:30

06:45

07:00

07:15

07:30

Errors

4:45

5:00

5:15

5:30

5:45

6:00

6:15

6:30

6:45

7:00

7:15

7:30

In alarm

OK

Insufficient data

Disabled/Muted actions

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Page | 10

4.8 WAF — API Protection

AWS WAF was configured and attached to the UniNotifyAPI prod stage. A Web ACL named UniNotifyWAF was created with two rules: the AWS Managed Core Rule Set which automatically blocks common web exploits including SQL injection and cross-site scripting, and a custom rate-based rule that blocks any IP address exceeding 100 requests within a five-minute window. WAF inspects every request before it reaches API Gateway, providing a security layer between the public internet and the Lambda function.

UniNotifyWAF

[↓ JSON](#)

Description

WAF protection for UniNotify API Gateway

ARN

 `arn:aws:wafv2:eu-no...`

Id

 `51285fc6-1c32-4...`[Manage details](#)[Manage rules](#)[Manage custom response bodies](#)[Manage resources](#)[View dashboard, logs and sampled requests](#)[View AI traffic analysis *new*](#)[Configure logging and sampled requests](#)

5. Frontend Application

The frontend is a React 18 single-page application built with Create React App, featuring a role-based login system with Teacher and Admin access. Teachers (teacher / teacher123) can send announcements to All, CE, CS, and Final Year audiences, while the Faculty option is restricted with a warning message. Admins (admin / admin123) have full access to all five audience groups, including Faculty.

The interface includes colored role badges, a user information card showing the logged-in user and access level, and a sign-out option that returns users to the login screen. Users can submit announcements through a simple form containing a title, audience dropdown, and message field. Axios sends the announcement to the API Gateway endpoint, while the UI displays success or error notifications based on the request result. The frontend is intentionally simple, with the project's main focus being the AWS cloud architecture and backend integration.

The screenshot displays the UniNotify application interface, which is divided into two main sections: a login/sign-in area on the left and a 'New Announcement' form on the right.

UniNotify GIKI COMMUNICATION PLATFORM

Sign in to your account
Select your role and enter your credentials to continue.

Role selection buttons: **Teacher** (selected) and **Admin**.

Information for Teachers: Teachers can send announcements to students only — CE, CS, Final Year, All Students.

Form fields for login: **USERNAME** (teacher) and **PASSWORD** (Enter your password).

Sign in as Teacher button.

New Announcement (TEACHER role badge)
Fill in the details and select your target audience.

TITLE field: e.g. Lab session cancelled — Thursday

AUDIENCE selection: **ALL** (selected), CE, CS, FY. A warning message states: **Faculty audience is restricted to Admins only.**

MESSAGE field (0 / 500 characters): Write your announcement here. Be clear and concise.

SEND ANNOUNCEMENT button.

Success Notification: ✓ Announcement sent to All Students!

Notification Card (AWS Notifications):

- From: AWS Notifications <no-reply@sns.amazonaws.com>
- To: u2023694
- Date: Mon 11/05/2026 5:11 PM
- Audience: All Students
- Message: Tomorrow morning lab cancelled
- Sent via UniNotify

6. Testing & Verification

The system was tested at three levels: unit testing of the Lambda function using the built-in Lambda test console, integration testing using PowerShell curl commands against the API Gateway endpoint, and end-to-end testing through the live CloudFront URL.

Test	Method	Expected Result	Status
Lambda unit test	Lambda console test event	Status 200, record in DynamoDB	Pass
API Gateway integration	PowerShell POST request	JSON success response	Pass
CORS validation	Browser form submission	No CORS errors in console	Pass
DynamoDB write	Explore items after submission	New item with UUID and timestamp	Pass
SNS email delivery	Check inbox after submission	Email arrives within 30 seconds	Pass
Audience routing	Submit for each audience option	Email to correct topic only	Pass
CloudWatch logging	Check log streams after test	Invocation log visible	Pass
WAF attachment	WAF console	UniNotifyAPI listed as resource	Pass

7. CLO Mapping

CLO	Description	How UniNotify Satisfies It
CLO 1	Understand core cloud computing services	Architecture covers IaaS, PaaS, SaaS, serverless, NoSQL, CDN, event-driven patterns across 9 AWS services
CLO 2	Analyze and apply cloud-based solutions	Designed and deployed a working multi-service serverless system solving a real problem with measurable outcomes
CLO 3	Develop practitioner skills on AWS	Hands-on configuration of Lambda, API Gateway, DynamoDB, SNS, S3, CloudFront, IAM, CloudWatch, and WAF

8. Challenges & Solutions

- CORS errors on first browser submission — resolved by enabling CORS on the API Gateway resource and ensuring the Lambda function returns Access-Control-Allow-Origin: * in every response header including error responses.
- SNS emails not arriving — caused by team members not clicking the AWS confirmation link. Resolved by re-sending confirmation emails from the SNS console and confirming all subscriptions before testing.
- Lambda unable to write to DynamoDB — caused by the IAM role not yet having DynamoDB permissions. Resolved by attaching the inline policy with the correct table ARN.
- CloudFront returning 403 Forbidden — caused by the S3 bucket policy not being updated after OAC creation. Resolved by pasting the CloudFront-generated bucket policy into S3 permissions.
- React app showing blank page on CloudFront — caused by uploading the public/ folder instead of the build/ folder. Resolved by running npm run build and re-uploading the correct folder.

9. Cost Analysis

All services used in UniNotify fall within AWS Free Tier limits for the duration of this project. The only exception is WAF which is not covered by Free Tier.

Service	Free Tier Limit	Our Usage	Cost
Lambda	1M requests/month	< 200 requests	\$0.00
DynamoDB	25GB storage + 25 RCU/WCU	< 1MB, < 10 writes	\$0.00
SNS	1,000 email notifications/month	< 50 emails	\$0.00
API Gateway	1M calls/month (12 months)	< 200 calls	\$0.00
S3	5GB storage (12 months)	~5MB	\$0.00
CloudFront	1TB transfer/month (12 months)	< 1MB	\$0.00
CloudWatch	10 metrics + 5GB logs free	Well within limits	\$0.00
WAF	Not Free Tier	1 Web ACL + 1 rule	~\$6/month
Total			~\$6/month

Note: WAF will be deleted immediately after the project demonstration to avoid ongoing charges. All other services will remain active at zero cost.

10. Conclusion

UniNotify successfully demonstrates a production-grade serverless architecture on AWS that solves a real problem for GIKI students and faculty. The project integrates nine AWS services into a cohesive, working application that covers the full breadth of CE308 course content — from SaaS and PaaS service models to REST API design, serverless computing, NoSQL databases, content delivery, cloud security, IAM, and monitoring.

The system performs reliably: announcements are stored permanently in DynamoDB, delivered via email within seconds through SNS, and the entire backend runs without any server management. The audience routing system ensures the correct recipients receive each announcement. CloudWatch provides full observability and WAF provides security hardening at the API layer.

This project gave the team hands-on experience with the AWS services that power real-world applications, and demonstrated how serverless architecture can deliver a complete, scalable solution with minimal cost and complexity compared to traditional EC2-based deployments.